ARCHITECTURE DESIGN DOCUMENT

AIOps PLE Platform

PLE + adaTT 기반 금융 상품 추천 시스템
기술 설계 ►►서 (Internal Reference)

Version 1.0 — 2026-04-01
대상 독자: 개발자, 아키텍트, 코드 리뷰어

> **Design vs Implementation Note**
> 본 문서는 **설계 의도와 목표 아키텍처**를 기술한다. 현재 구현체는 설계의 부분 집합일 수 있으며, 아직 구현되지 않은 컴포넌트가 포함되어 있다. 구현 현황은 코드베이스와 `pipeline_state.json`을 참조한다.

목차

1. 시스템 개요 4
   1.1. 시스템 목적 4
   1.2. 대상 사용자 및 운영 환경 4
   1.3. On-Prem vs AWS 대비 4
2. 설계 철학 5
   2.1. 원칙 1: 견고한 설명 (Inherent Explainability) 5
   2.2. 원칙 2: 안정적 내결함성 (Graceful Degradation) 5
   2.3. 원칙 3: 유연한 확장성 (Extensibility) 5
   2.4. 원칙 4: 통합 관리 가능성 (Manageability) 5
3. 아키텍처 결정 이력 6
   3.1. ALS에서 Black-Litterman으로 6
   3.2. Black-Litterman 드랍 사유 6
   3.3. PLE + adaTT 선택 과정 6
   3.4. 결정 흐름 요약 7
4. PLE + 이종 전문가 Basket 8
   4.1. 기존 PLE의 한계와 이종 전문가 도입 8
   4.2. Pool / Basket / Runtime 3계층 8
   4.3. Expert Pool — 11종 등록 목록 8
   4.4. 7종 Expert 선택 근거 9
      4.4.1. Temporal Ensemble: Expert 내부 앙상블 9
   4.5. CGC Layer + Attention 9
   4.6. Dual-Registry 아키텍처 9
5. adaTT + Task Group 11
   5.1. 4개 금융 DNA 그룹 11
   5.2. Adaptive Task Transfer (adaTT) 11
   5.3. Loss-Level 전이: Logit Transfer (3-Method Dispatch) 11
   5.4. HMM Triple-Mode Projection 11
   5.5. Multidisciplinary Per-Task Routing 12
6. Feature Pipeline 13
   6.1. 5축 피처 분류 (5-Axis Feature Classification) 13
   6.2. 12개 Feature Group (316D) 13
      6.2.1. 4개 Base Group (transform 타입) 13
      6.2.2. 8개 Generated Group (generate 타입) 13
   6.3. 11개 학문 분야 (Multidisciplinary Feature Design) 13
   6.4. 피처의 이중 역할: 학습 + 추천사유 14
   6.5. 3단계 정규화 (Normalization Pipeline) 14

7. Training Pipeline 15
7.1. Phase 0: Data Preparation 15
7.1.1. 데이터 리키지 4중 방지 15
7.1.2. 데이터 처리 백엔드 정책 15
7.2. Phase 1–3: Ablation Study (48 Scenarios) 15
7.3. 모델 학습 (Phase 1–3 공통) 16
7.3.1. Per-Task Loss Dispatch 16
7.3.2. Uncertainty Weighting (Kendall et al.) 16
7.3.3. Evidential Deep Learning (Config-Gated) 16
7.3.4. SAE Regularization (Detached, Config-Gated) 16
8. 증류 + 서빙 (Distillation + Serving) 17
8.1. Phase 4: Knowledge Distillation (PLE → LGBM) 17
8.2. Phase 5: Serverless Serving (Lambda) 17
8.2.1. 서빙 아키텍처 17
8.2.2. FD-TVS Composite Scoring 17
8.2.3. DNA Modifier 17
8.2.4. Constraint Engine 17
8.2.5. 추천사유 생성 파이프라인 18
9. 모니터링 + 컴플라이언스 19
9.1. 파이프라인 추적 19
9.2. Per-Stage 체크포인트 19
9.3. Audit Artifacts 19
9.4. 해석 가능성 파이프라인 (Stage 8.5) 19
9.5. 암호화 파이프라인 (Stage 3) 20
10. End-to-End Data Flow 21
10.1. 전체 데이터 흐름도 21
10.2. 모델 내부 데이터 흐름 22
10.3. GPU/CPU 가속 매핑 23
11. 부록: PLEModel 빌드 자동화 24
11.0.1. PLEInput 데이터 컨테이너 24
12. 부록: Config 파일 구조 24

시스템 개요

시스템 목적

AIOps PLE Platform은 **금융 상품 추천**을 위한 end-to-end 멀티태스크 학습 플랫폼이다. 은행, 카드사, 금융지주사를 대상으로 예금, 카드, 대출, 투자 등 종합 금융 상품에 대한 개인화 추천과 그 **설명 가능한 근거**를 동시에 생성한다.

AI 추천의 최종 산출물은 확률(0.73)이 아니라 **고객이 납득할 수 있는 이유**이다. 설득의 대상은 항상 사람이며, 세 가지 수준의 설명이 필요하다:

대상	질문	기대 수준
고객	"왜 이 상품인가"	신뢰 → 전환
행원	"왜 이 고객에게 이걸 권하는가"	영업 근거
금감원	"왜 이런 결정을 내렸는가"	규제 준수 (EU AI Act, 금감원 가이드라인)

대상 사용자 및 운영 환경

- **금융사 ML 운영 인력**: 1–2명 현실. 모델 N개 × 설명 모듈 × 앙상블 로직 = 관리 포인트 N개가 아닌, 단일 config 체계로 통합 관리가 가능해야 한다.
- **하드웨어**: GPU 1–4장 수준. MLP 파라미터 확장이 불가능하므로 구조적 편향(inductive bias)으로 표현력을 확보한다.
- **인프라**: On-Prem (Airflow + DuckDB) 또는 AWS (SageMaker + S3). 아키텍처 철학은 동일하게 유지하며 인프라만 전환한다.

On-Prem vs AWS 대비

관점	On-Prem (현재)	AWS (목표)	전환 근거
오케스트레이션	Airflow 86 DAGs ($300/월)	Step Functions 5개 ($0/월)	실행당 과금
학습	로컬 GPU	SageMaker Spot (70% 절감)	병렬 ablation
저장소	DuckDB 파일	S3 Parquet	내구성, IAM
서빙	FastAPI + Docker	Lambda / ECS Fargate	규모별 자동 전환
실험 관리	MLflow (Docker)	SageMaker Experiments	서버 유지 비용 제거
모니터링	커스텀 drift_monitor	SageMaker Model Monitor	관리형 서비스

설계 철학

4가지 핵심 설계 원칙이 모든 기술적 결정의 출발점이다.

원칙 1: 견고한 설명 (Inherent Explainability)

사후적 설명(post-hoc SHAP/LIME)이 아닌 **구조적 설명**을 지향한다.

- **SHAP/LIME의 한계**: 모델과 분리되어 설명이 내부 동작과 괴리. 입력이 약간 변해도 설명이 크게 바뀜. 추론마다 별도 계산으로 서빙 latency 수 배 증가.
- **구조적 접근**: 모델 구조(gate, evidential, contrastive) 자체에서 설명이 산출. forward pass 한 번에 추론 + 설명 동시 생성.

이종 전문가(Heterogeneous Expert) 구조가 이를 가능하게 한다:

- CGC gate weight 자체가 설명: "Temporal(0.35), DeepFM(0.28), HGCN(0.22)"
- expert 이름이 곧 비즈니스 맥락: "시계열 패턴 35% 기여, 피처 교차 28%, 상품 계층 22%"
- 동종 MLP expert의 경우 "MLP 3번이 기여했습니다"는 비즈니스 의미가 없음

원칙 2: 안정적 내결함성 (Graceful Degradation)

expert 하나가 쓸모없더라도 나머지가 gate 재분배로 자연스럽게 지탱한다.

- ablation으로 "어떤 expert를 빼도 급격한 성능 저하 없음"을 증명
- 금융사 특성: 빅테크의 "AUC 0.01 올리면 매출 N억" 공격적 실험과 달리, "모델 이상 작동 시 금감원 제재" → 보수적 운영, 안정성 우선

원칙 3: 유연한 확장성 (Extensibility)

새 피처/태스크 추가 시 config만 변경. 기존 구조 수정 없이 adaTT가 새 관계를 자동 학습.

- **Pool/Basket/Runtime 3계층**: 코드(Pool) → Config(Basket) → 학습(Runtime) 분리
- 도메인 전환 시 코드 수정 0: config 파일만 `configs/financial/` → `configs/ecommerce/`로 교체

원칙 4: 통합 관리 가능성 (Manageability)

전체 파이프라인(피처 생성 → 학습 → 증류 → 서빙 → 추천사유)이 하나의 config 체계(`pipeline.yaml` + `feature_groups.yaml`)로 통합 관리.

- **Config-Driven**: YAML 하나로 데이터/태스크/모델/인프라 정의. 코드 변경 없이 새 문제 적용.
- **Registry Pattern**: Expert, Task, Feature, Model, Tower 모두 플러그인 방식 등록.
- **Schema-First**: 데이터 스키마가 파이프라인 전체를 결정.

아키텍처 결정 이력

ALS에서 Black-Litterman으로

기존 금융 상품 추천 시스템은 ALS(Alternating Least Squares) 기반 협업 필터링이었다. MLOps 도입을 결정하면서 차세대 모델 검토가 시작되었다.

초기 후보:
1. DL 모델군 (DeepFM, Wide&Deep, AutoInt)
2. GBM 모델군 (XGBoost, LightGBM, CatBoost)
3. DL + GBM 앙상블

앙상블 검토 중 금융 포트폴리오 최적화의 **Black-Litterman 모델**을 추천에 적용하려는 시도가 있었다. 각 모델의 예측을 "전문가 의견(view)"으로 취급하고, 불확실성(리스크)에 비중을 두어 베이지안 업데이트로 통합하는 구조를 구상했다.

Black-Litterman 드랍 사유

설계 단계에서 핵심 한계가 확인되었다:

한계	상세
비즈니스 해석 불가	베이지안 업데이트 과정에서 각 모델의 기여가 혼합 → "왜 이 상품을 추천했는가" 설명 어려움. 금감원 규제 환경에서 치명적.
구조적 불일치	금융 포트폴리오(연속 비중 배분)와 상품 추천(이산 선택)의 근본적 차이
View matrix 자동화 난이도	각 모델의 불확실성 추정이 주관적이고 자동화 어려움
멀티태스크 한계	이탈/추천/세그먼트/가치 등을 하나의 BL 프레임워크로 통합 시 태스크 간 관계 표현 수단 부족

PLE + adaTT 선택 과정

질문을 재구성했다: "여러 모델을 어떻게 잘 섞을 것인가"(앙상블) → "하나의 모델 안에서 여러 전문가가 협업하는 구조는 없는가"(MoE/PLE).

MTL(Multi-Task Learning) 계열 검토:

구조	특성	한계
Shared-Bottom	전문가 1개를 전 태스크가 공유	negative transfer 심각
MMoE	전문가 N개 + 태스크별 gate	모든 expert가 모든 태스크에 노출 → 전문화 부족
PLE	shared + task-specific expert 분리, CGC gate	선택됨

PLE가 선택된 이유:
- expert network = 내부 앙상블 (전문가별 특화)
- CGC gate = 태스크별 전문가 가중치 → 설명 가능성
- 단일 모델 학습/배포 → 관리 포인트 1개, 서빙 비용 고정
- Black-Litterman이 외부에서 하려던 것(전문가 의견의 불확실성 기반 통합)을 모델 내부 구조로, 데이터 기반으로 해결

결정 흐름 요약

```
ALS (기존)
  ↓ "MLOps 도입, 차세대 모델 필요"
DL + GBM 앙상블 검토
  ↓ "단순 앙상블로는 멀티태스크 통합 어려움"
Black-Litterman 시도
  ↓ "금융 포트폴리오 ≠ 상품 추천, 설계 단계에서 드랍"
PLE + adaTT 선택
  ↓ "온프렘에서 프로토타입 검증"
AWS 마이그레이션
  ↓ "ablation 병렬화 + end-to-end 서빙 필요"
벤치마크 데이터 + Ablation 실행
  ↓ (예정)
증류 + Lambda 서빙
```

PLE + 이종 전문가 Basket

기존 PLE의 한계와 이종 전문가 도입

기존 PLE (Tencent, 2020)의 shared expert는 **동종 MLP × N개** (같은 구조, 다른 초기화)로 구성된다. 파라미터를 늘려야 표현력이 올라가므로 경량 모델에서 표현력이 부족하다.

본 시스템의 접근: shared expert = **이종 전문가 7개**. 각 expert가 다른 inductive bias로 같은 데이터를 다른 관점에서 처리하고, CGC gate가 "이 태스크에는 어떤 관점이 유용한가"를 학습한다. 단일 대형 MLP보다 가벼우면서 표현력은 더 높다.

Pool / Basket / Runtime 3계층

```
Pool (등록 목록)        Basket (Config 선택)       Runtime (가중 실행)

┌──────────────┐     ┌──────────────┐       ┌──────────────┐
│ 모든 구성 요소가  │     │ YAML config으로   │       │ 실행 시점에       │
│ Registry에 등록  │──▶│ 부분 집합 선택    │──▶│ 가중치 기반 결합  │
│ (플러그인 방식)   │     │ (도메인별 교체)   │       │ (학습/추론)       │
└──────────────┘     └──────────────┘       └──────────────┘

     코드 영역              Config 영역              모델 영역
```

계층	역할	변경 주체
Pool	사용 가능한 모든 구성 요소를 Registry에 등록	개발자 (코드 추가)
Basket	특정 파이프라인에 사용할 부분 집합을 YAML로 선택	운영자 (config 교체)
Runtime	선택된 구성 요소를 가중 결합하여 실행	모델 (학습 중 자동)

Expert Pool — 11종 등록 목록

#	등록 이름	클래스	주요 Axis	Basket
1	`deepfm`	DeepFM Expert	State (피처 상호작용)	O
2	`temporal_ensemble`	Temporal Ensemble	Timeseries (Mamba+LNN+Transformer)	O
3	`hgcn`	Unified HGCN	Hierarchy (쌍곡 그래프)	O
4	`perslay`	PersLay Expert	Snapshot (TDA global)	O
5	`causal`	Causal Expert	Snapshot (NOTEARS DAG)	O
6	`lightgcn`	LightGCN Expert	Item (그래프 협업 필터링)	O
7	`optimal_transport`	OT Expert	Snapshot (Sinkhorn)	O
8	`mlp`	MLP Expert	State (기본)	—
9	`mamba`	Mamba Expert	Timeseries (SSM)	—

#	등록 이름	클래스	주요 Axis	Basket
10	`autoint`	AutoInt Expert	State (Self-Attention)	—
11	`xdeepfm`	XDeepFM Expert	State (CIN + Deep)	—

7종 Expert 선택 근거

각 expert는 서로 중복되지 않는 inductive bias를 가진다:

Expert	Inductive Bias	적은 파라미터로 잡는 패턴
DeepFM	feature interaction	피처 간 2차 교차 효과
Temporal Ensemble	복합 시계열 (Mamba+LNN+Transformer)	장/단기/단절 시계열 통합
HGCN	계층 구조 (쌍곡 공간)	상품 카테고리 트리
PersLay	위상 구조	TDA persistence 형상
LightGCN	그래프 관계	고객-상품 협업 필터링
Causal	인과 관계	인과 방향 (DAG 제약)
Optimal Transport	분포 매칭	고객 세그먼트 간 분포 차이

Temporal Ensemble: Expert 내부 앙상블

금융 시계열은 단일 모델로 포착할 수 없는 복합 구조를 가진다. Temporal Expert 자체를 3개 모델의 내부 앙상블로 설계했다:

모델	강점	약점
Mamba (SSM)	장기 의존성, O(n) 효율	불규칙 간격에 약함
LNN (Liquid NN)	단절/불규칙 적응	긴 맥락 요약 약함
Transformer	단기 맥락 추출	$O(n^2)$, 긴 시퀀스 비효율

CGC Layer + Attention

`CGCLayer`는 PLE의 핵심 빌딩 블록이다. 각 태스크에 대해 gating network가 shared + task-specific expert 출력을 결합한다.

- **dim_normalize=True**: expert 출력 차원이 다를 때 `sqrt(mean_dim/dim)` 스케일링으로 차원 불균형 보정
- **bias_high/bias_low**: domain-relevant expert에 초기 바이어스 주입
- **entropy regularization**: expert collapse 방지

Stacked PLE: 3개 CGC layer. Layer 0에서 이종 Expert Basket + FeatureRouter를 사용하고, Layer 1–2는 동종 MLP expert로 추상화한다.

Dual-Registry 아키텍처

```
Expert Pool Registry (core.model.experts.registry.ExpertRegistry)
    └── AbstractExpert(input_dim, config)
    └── 11종 등록

Expert PLE Registry (core.model.ple.experts.ExpertRegistry)
    └── BaseExpert(input_dim, output_dim, dropout)
    └── CGCLayer 기본 expert 생성용
```

```
Expert Basket (core.model.ple.experts.ExpertBasket)
    └── Pool Registry → Basket 선택 → CGCLayer.shared_experts에 주입
```

adaTT + Task Group

4개 금융 DNA 그룹

초기 설계에서는 GMM 클러스터별 태스크 서브헤드를 두려 했으나, K=20, T=18이면 서브헤드 360개 → 관리 불가, 과적합 위험. 방향을 전환하여 **태스크를 금융적 DNA 관점으로 그룹핑**했다:

Group	금융 DNA	포함 태스크	intra	inter
engagement	고객이 반응하는가	has_nba, engagement_score, next_mcc, top_mcc_shift	0.8	0.3
lifecycle	고객이 어디에 있는가	churn_signal, product_stability, tenure_stage, segment_prediction	0.7	0.3
value	고객이 얼마나 가치있는가	spend_level, income_tier, mcc_diversity_trend	0.6	0.3
consumption	고객이 무엇을 살 것인가	nba_primary, cross_sell_count, will_acquire_* (5개)	0.7	0.3

총 **18개 태스크**가 4개 의미 그룹으로 구성된다.

Adaptive Task Transfer (adaTT)

- **Intra-group**: 같은 그룹 내 태스크 간 강한 전이 (0.6–0.8)
- **Inter-group**: 다른 그룹 간 약한 전이 (0.3) — 간섭 최소화
- **Negative transfer threshold**: 성능 저하 시 전이 자동 차단
- **EMA decay**: 전이 가중치 안정화
- **Warmup/freeze epochs**: 초기 안정화

Loss-Level 전이: Logit Transfer (3-Method Dispatch)

태스크 간 명시적 인과 관계를 5개 엣지로 모델링한다:

Source	Target	Method	의미
engagement_score	has_nba	output_concat	활성도 → 가입 (선행지표)
has_nba	nba_primary	output_concat	가입 여부 → 어떤 상품
churn_signal	product_stability	output_concat	이탈 → 상품 안정성
spend_level	cross_sell_count	output_concat	소비수준 → 교차판매
next_mcc	nba_primary	hidden_concat	다음 업종 → 다음 상품 (feature sharing)

3가지 전이 방법:

- **residual**: source output → Linear → tower_dim, 잔차 합산
- **output_concat**: source output과 tower input concat → Linear → tower_dim
- **hidden_concat**: source pre-tower hidden과 tower input concat → Linear → tower_dim

`logit_transfer_strength: 0.5` — 전이 비율.

HMM Triple-Mode Projection

3개 HMM 모드를 태스크 그룹별로 라우팅한다:

Task Group	HMM Mode	의미
engagement	behavior	행동 모드 → 반응/활성
lifecycle	lifecycle	생애주기 모드 → 이탈/유지
value	journey	여정 모드 → 가치/소비
consumption	journey	여정 모드 → 소비 패턴

각 모드별 16D → `task_expert_output_dim`으로 projection 후 tower input에 additive fusion.

Multidisciplinary Per-Task Routing

24D multidisciplinary 피처를 4개 태스크 그룹에 6D씩 라우팅한다:

Task Group	학제	차원
engagement	chemical_kinetics	[0:6]
lifecycle	epidemic_diffusion	[6:12]
value	crime_pattern	[12:18]
consumption	interference	[18:24]

Feature Pipeline

5축 피처 분류 (5-Axis Feature Classification)

모든 피처를 5개 축으로 분류하고, 각 축에 대응하는 Feature Generator와 Expert를 매핑한다.

축	시간 의존성	변화 속도	대표 데이터	처리 방식
State	없음 (정적)	거의 불변	나이, 성별, 가입일	피처 상호작용 (FM)
Snapshot	장기 (월/분기)	느림	12개월 거래 위상, HMM 상태	장기 패턴 추출
Timeseries	단기 (일/주)	빠름	최근 90일 거래 시퀀스	시퀀스 모델링 (SSM)
Hierarchy	없음 (구조적)	느림	MCC 코드 계층, 상품 카테고리	쌍곡 임베딩
Item	없음 (관계적)	중간	고객-상품 상호작용	그래프 협업 필터링

12개 Feature Group (316D)

4개 Base Group (transform 타입)

기존 raw 컬럼을 변환하여 생성. Generator가 필요 없다.

- `base_rfm`: RFM 기반 피처 (quantile transform)
- `base_demographics`: 인구통계 피처
- `base_product`: 상품 보유 현황
- `base_activity`: 활동 지표

8개 Generated Group (generate 타입)

전용 Feature Generator가 피처를 생성한다.

Group	Generator	출력 D	설명
tda_topology	tda_extractor	70D	Persistence Diagram 기반 위상 분석
hmm_states	hmm_triple_mode	48D	journey/lifecycle/behavior 상태 추정
hyperbolic_embedding	hyperbolic_embedding	20D	쌍곡 공간 계층 구조 임베딩
temporal_pattern	temporal_pattern	가변	시계열 집계 + 주기 인코딩
multidisciplinary	multidisciplinary	24D	화학동역학, 전염병확산, 간섭, 범죄패턴
gmm_clustering	gmm	가변	Soft posterior probabilities
economics	economics_extractor	17D	MPC, 소득 탄력성, 항상소득
model_derived	model_feature_extractor	27D	이전 모델 출력 활용

11개 학문 분야 (Multidisciplinary Feature Design)

학문 분야	도입 요소	금융 고객에서의 의미
위상수학	TDA Persistent Homology	소비 패턴의 구조적 형태
쌍곡기하학	Hyperbolic GCN	상품 계층 구조의 왜곡 없는 임베딩
확률과정	HMM 상태 전이	고객 생애주기 단계 추적
제어이론	Mamba (State Space Model)	장기 행동 의존성

학문 분야	도입 요소	금융 고객에서의 의미
경제학	항상소득/일시소득, 한계효용	소비 여력의 구조적 분해
금융공학	리스크 지표, Bandit 탐색/활용	상품 탐색 성향
그래프이론	LightGCN	유사 고객군 행동 전이
통계학	GMM 군집화	연성 세그먼테이션
인과추론	Causal DAG (NOTEARS)	행동 간 인과 방향
최적수송	Sinkhorn Optimal Transport	세그먼트 간 분포 이동
신경미분방정식	Liquid Neural Network	불규칙 시간 간격 적응

핵심: 이 관점들은 서로 중복되지 않는다. 위상수학의 "형태"와 경제학의 "효용"은 같은 고객의 완전히 다른 측면이며, 각각이 독립적으로 기여한다 (ablation으로 증명).

피처의 이중 역할: 학습 + 추천사유

다학제 피처의 가치는 학습 성능(AUC 기여)만이 아니다. ablation에서 TDA 피처를 빼도 AUC가 0.01밖에 안 떨어질 수 있지만, "고객님의 소비 패턴이 지속적으로 안정적인 형태를 보이고 있어"라는 설명은 TDA 없이는 생성 불가하다.

모든 피처에 **비즈니스 맥락 역매핑**을 구축:

- `hmm_lifecycle_prob_growing` → "성장 단계 고객"
- `mamba_temporal_d3` → "최근 3개월 소비 증가 추세"
- `hgcn_hierarchy_d5` → "투자 상품군과 가까운 포지션"

3단계 정규화 (Normalization Pipeline)

```
Stage 1: 멱법칙 감지 (skew+kurt → log-log R²) + log1p 복사본 생성
Stage 2: StandardScaler (continuous 컬럼만, binary 제외, TRAIN fit only)
Stage 3: 멱법칙 _log 복사본은 스케일링하지 않음 (raw magnitude 보존)
```

- Scaler는 **TRAIN split에서만 fit**. val/test는 train에서 fit된 scaler로 transform만.
- Binary 컬럼은 스케일링에서 제외.

Training Pipeline

Phase 0: Data Preparation

10+ Stage PipelineRunner가 raw data에서 training-ready tensor까지 변환한다.

Stage	이름	설명	GPU
1	DataAdapter	DuckDB-native raw data load, AdapterRegistry 기반	—
1.5	TemporalPrep	시퀀스 절단 (drop last month), prod_* 재계산	—
2	SchemaClassifier	모든 컬럼을 5-Axis로 분류	—
3	EncryptionPipeline	PII → SHA256 → INT32 (도메인별 salt)	—
4	FeatureGroupPipeline	8개 Generator 실행 + PowerLawAwareScaler	cuDF
5	LabelDeriver	Config-driven 18개 레이블 생성	—
5.5	LeakageValidator	시퀀스/상관관계/제품/시간 4중 누수 검증	—
6	SequenceBuilder	flat → 3D tensor (event_seq, session_seq)	—

Phase 0는 **CPU 인스턴스**에서 실행. GPU 인스턴스를 Phase 0에 낭비하지 않는다.

데이터 리키지 4중 방지

1. **시퀀스 절단**: 17개월 → 16개월 (drop last month)
2. **제품 재계산**: month 16 state 기준으로 prod_* 재계산
3. **시간 기반 분할**: temporal split + gap_days (최소 7일)
4. **LeakageValidator**: 시퀀스/상관관계/제품/시간 검증, >0.95 상관 피처 자동 제거

데이터 처리 백엔드 정책

우선순위: cuDF (GPU) → DuckDB (CPU columnar) → pandas (최후 fallback, 10K 이하만). `pd.read_parquet()`, `pd.concat()`, `df.apply()` 등 pandas 직접 사용을 지양한다.

Phase 1–3: Ablation Study (48 Scenarios)

4개 차원 × 다수 시나리오로 체계적 실험을 수행한다:

Phase	차원	내용	Job 수
1	Feature Group	full + base_only + bottom-up + top-down	16
2	Expert	deepfm baseline + bottom-up + top-down + mlp_only	16
3	Task × Structure	4 tiers × 4 structures (shared_bottom / ple_only / adatt_only / full)	16

- **Bottom-up**: base + X → 독립 기여 측정
- **Top-down**: full – X → irreplaceability 측정

모델 학습 (Phase 1–3 공통)

Per-Task Loss Dispatch

Loss Type	Module	용도	Task Type
focal	FocalLoss(alpha, gamma)	불균형 이진 분류 (calibrated alpha)	binary
huber	SmoothL1Loss	이상치 강건 회귀	regression
mse	MSELoss	기본 회귀	regression
ce	CrossEntropyLoss(weight)	다중 클래스 (auto class_weights)	multiclass
infonce	InfoNCELoss(temperature)	대조 학습	contrastive

AMP (Mixed Precision) 환경에서 tower output을 FP32로 cast 후 loss 계산 (overflow 방지).

Uncertainty Weighting (Kendall et al.)

$$L_{\text{total}} = \sum_k \left[\exp(-s_k) \cdot L_k + \frac{s_k}{2} \right]$$

s_k: 태스크 k의 learnable log-variance. 불확실성 높은 태스크 → 자동 가중치 감소.

Evidential Deep Learning (Config-Gated)

Task Type	분포	파라미터	불확실성
Binary	Beta(alpha, beta)	alpha, beta	1/(alpha+beta)
Multiclass	Dirichlet(alpha)	alpha_1..K	K/sum(alpha)
Regression	Normal-Inverse-Gamma	mu, v, alpha, beta	beta/(v*(alpha-1))

SAE Regularization (Detached, Config-Gated)

Anthropic-style Sparse Autoencoder. shared expert concatenated output에 적용. **detached**로 메인 모델 gradient에 영향 없음 (분석용 sidecar). Tied weights, pre-bias centering, ReLU activation.

증류 + 서빙 (Distillation + Serving)

Phase 4: Knowledge Distillation (PLE → LGBM)

```
PLE Teacher (GPU 학습)
    ↓ Soft Labels 생성 (temperature=5.0)
    ↓ S3에 저장
LGBM Student (CPU 학습)
    ↓ alpha=0.3 (30% hard + 70% soft)
    ↓ IG 기반 피처 선택 (top-k features)
    ↓ 경량 모델 저장
서빙 (실시간: LGBM ~5ms, 배치: PLE)
```

- **Temperature**: 5.0 — soft label의 정보량 극대화
- **Alpha**: 0.3 — hard label 30% + soft label 70% 혼합
- **Fidelity validation**: teacher-student 예측 일치도 검증

Phase 5: Serverless Serving (Lambda)

서빙 아키텍처

- **실시간**: LGBM student → Lambda (~5ms latency)
- **배치**: PLE teacher → SageMaker Batch Transform
- **규모 전환**: Lambda (기본) ↔ ECS Fargate (대규모) 자동 전환

FD-TVS Composite Scoring

태스크별 예측을 단일 추천 스코어로 통합한다:

태스크	가중치
has_nba	0.20
nba_primary	0.30
cross_sell_count	0.15
churn_signal	0.15
product_stability	0.10
engagement_score	0.10

DNA Modifier

세그먼트별 가중치 조정: TOP(1.3), PARTICULARES(1.0), UNIVERSITARIO(0.8), UNKNOWN(0.7).

Constraint Engine

제약	설명
Fatigue	7일 내 최대 5회 메시지
Eligibility	min_score > 0.05, max_churn_prob < 0.6
Owned Product	prod_* prefix로 이미 보유한 상품 제외

제약	설명
Product Tier	standard 3개월, growth 6개월, premium 12개월 최소 가입기간
Top-K Diversity	MMR (lambda=0.5)로 다양성 보장

추천사유 생성 파이프라인

```
[모델 추론] → [IG Attribution: 상위 피처] → [비즈니스 역매핑]
→ [LLM Agent: 맥락 조합 → 자연어 추천사유]
```

4가지 설명 수준이 동시 제공된다:

1. **gate weight**: "어떤 관점이 기여" (expert-level)
2. **contrastive**: "왜 이것이고 저것은 아닌가" (대조)
3. **evidential**: "얼마나 확실한가" (불확실성 정량화)
4. **SAE**: "내부적으로 어떤 개념이 활성화됐는가" (뉴런-level)

모니터링 + 컴플라이언스

파이프라인 추적

Artifact	위치	용도
`pipeline_manifest.json`	output_dir/	전체 파이프라인 config 스냅샷
`pipeline_state.json`	output_dir/	Stage별 완료/실패 상태, resume 지원
`feature_stats.json`	output_dir/	zero-variance, NaN 비율, 피처 컬럼 수
`label_stats.json`	output_dir/	class balance, positive rate

Per-Stage 체크포인트

Stage	Artifact	형식
Feature	`features.parquet`	Parquet
Label	`labels.parquet`	Parquet
Sequence	`sequences.npy`, `seq_lengths.npy`	NumPy
Scaler	`scaler_params.json`	JSON
Leakage	`leakage_report.json`	JSON

Audit Artifacts

```
audit/
├── schema/          ← 스키마 검증 결과
├── encryption/      ← PII 처리 감사 로그
├── scaler/          ← scaler_params.json
├── labels/          ← label_transforms.json
├── leakage/         ← LeakageValidator 결과
└── fidelity/        ← 증류 fidelity 검증
```

해석 가능성 파이프라인 (Stage 8.5)

분석	목적	출력
Integrated Gradients (IG)	피처 기여도 측정	attribution scores
Expert Redundancy CCA	Expert 간 중복성 검출	CCA correlation matrix
CGC Gate Analysis	태스크별 Expert 가중치 분석	attention heatmap
HGCN Interpretable	계층 구조 설명	hierarchy paths
Multi Interpreter	다학제 해석 통합	structured reasons
Template Reason Engine	자연어 추천 사유	text templates
XAI Quality Evaluator	설명 품질 평가	quality scores
Model Card	모델 문서 자동 생성	model_card.json

암호화 파이프라인 (Stage 3)

```
Schema (pii: true)
    ↓ derive_from_schema()
EncryptionPolicy (per source, per column)
    ↓
EncryptionPipeline.process_source()
    ├── Step 1: Drop (phone, email, SSN 등)
    ├── Step 2: SHA256 Hash (domain-specific salt)
    ├── Step 3: Integer Index (hash → INT32)
    └── Step 4: Audit report
```

16개 PII 도메인 정의 (CUSTOMER, ACCOUNT, CARD, MERCHANT 등). SaltManager가 AWS Secrets Manager 또는 로컬에서 도메인별 salt를 관리한다.

End-to-End Data Flow

전체 데이터 흐름도

```
┌──────────────────────────────────────────────────────────────────────┐
│                    Phase 0: Data Preparation                         │
│                                                                      │
│  S3 Raw Parquet                                                      │
│       │                                                              │
│       ▼                                                              │
│  Stage 1: DataAdapter (DuckDB-native load)                           │
│       │                                                              │
│       ▼                                                              │
│  Stage 1.5: TemporalPrep (시퀀스 절단, prod 재계산)                  │
│       │                                                              │
│       ▼                                                              │
│  Stage 2: SchemaClassifier (5-Axis 분류)                             │
│       │                                                              │
│       ▼                                                              │
│  Stage 3: EncryptionPipeline (PII → SHA256 → INT32)                  │
│       │                                                              │
│       ▼                                                              │
│  Stage 4: FeatureGroupPipeline (8 Generators + Normalization)        │
│       │         output: features.parquet (316D)                      │
│       ▼                                                              │
│  Stage 5: LabelDeriver (18 tasks, config-driven)                     │
│       │         output: labels.parquet                               │
│       ▼                                                              │
│  Stage 5.5: LeakageValidator (4-check, auto-drop)                    │
│       │                                                              │
│       ▼                                                              │
│  Stage 6: SequenceBuilder (flat → 3D tensors)                        │
│              output: event_seq.npy, session_seq.npy                  │
└──────────────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────────────┐
│                Phase 1-3: Ablation Training                          │
│                                                                      │
│  Stage 7: DataLoader (temporal split, gap_days)                      │
│       │                                                              │
│       ▼                                                              │
│  Stage 8: PLETrainer                                                 │
│       │   PLE (3-layer CGC, 7 shared + 1 task expert)                │
│       │   adaTT (4 groups, intra/inter transfer)                     │
│       │   Logit Transfer (5 edges, 3 methods)                        │
│       │   HMM Triple-Mode + Multidisciplinary Routing                │
│       │   Evidential + SAE (config-gated)                            │
```

```
│       │   Per-task Loss + Uncertainty Weighting                  │
│       │   AMP FP16 forward + FP32 loss                           │
│       │                                                          │
│       ▼                                                          │
│  Stage 8.5: Model Analysis                                       │
│       │   IG, CCA, Gate, HGCN, Multi Interpreter                 │
│       │   Template Engine, XAI Quality, Model Card               │
└──────────────────────────┬──────────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────────┐
│                Phase 4: Distillation                             │
│                                                                  │
│  Stage 9: StudentTrainer                                         │
│       │   PLE teacher → LGBM students                            │
│       │   Soft label (T=5.0, alpha=0.3)                          │
│       │   IG 기반 피처 선택 + fidelity validation                 │
└──────────────────────────┬──────────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────────┐
│                Phase 5: Serving                                  │
│                                                                  │
│  Stage 9.5: Context Vector Store (RAG embedding)                 │
│       │                                                          │
│       ▼                                                          │
│  Stage 10: CPE + Agentic Reason Orchestrator                     │
│       │   FD-TVS scoring + DNA modifier                          │
│       │   Constraint Engine (fatigue, eligibility, diversity)     │
│       │   L1+L2a+L2b 추론 체인                                    │
│       │                                                          │
│       ▼                                                          │
│  Lambda / ECS Fargate (서버리스 서빙)                              │
│       output: 추천 상품 + 자연어 추천사유 + 불확실성 정량화          │
└──────────────────────────────────────────────────────────────────┘
```

모델 내부 데이터 흐름

```
5-Axis Features (316D)
    │
    ▼ FeatureRouter (축별 분배)
┌──────────────────────────────────┐
│  Expert Basket (7 shared)        │
│  DeepFM ← State축               │
│  Temporal ← Timeseries축        │
│  HGCN ← Hierarchy축             │
│  PersLay ← Snapshot축           │
│  Causal ← Snapshot축            │
│  LightGCN ← Item축              │
│  OT ← Snapshot축                │
└──────────┬──────────────────────┘
           │
```

```
▼ CGC Layer × 3 (dim_normalize, entropy reg.)
│
▼ HMM Triple-Mode Projection (16D × 3 modes)
│
▼ Multidisciplinary Routing (24D → 4 × 6D)
│
▼ adaTT (intra 0.6-0.8, inter 0.3)
│
▼ Logit Transfer (5 edges, strength=0.5)
│
▼ Task Towers × 18 (TowerRegistry)
│
├── Evidential Layer (regression tasks)
├── SAE Regularization (detached sidecar)
└── Per-task Loss + Uncertainty Weighting
```

GPU/CPU 가속 매핑

Stage	대상	CPU 경로	GPU 경로
1	데이터 로딩	DuckDB (primary)	cuDF (선택적)
4	Generator 실행	pandas fallback	cuDF primary (cuML for GMM)
4	TDA persistence	ripser (NumPy)	cuPY + ripser (5–10x)
4	StandardScaler	NumPy	cuPY (3–5x on 100M+ rows)
7	Training 데이터 로딩	PyArrow (zero-copy)	—
8	Model training	PyTorch CPU	PyTorch CUDA + AMP

GPU 가속은 선택적이며, cuDF/cuPY 미설치 시 CPU 경로로 자동 폴백한다.

부록: PLEModel 빌드 자동화

`PLEModel.__init__(config: PLEConfig)` 호출 시 다음 순서로 자동 빌드된다:

1. `_build_extraction_layers()` — Stacked CGC + FeatureRouter
2. `_build_cgc_attention()` — Per-task attention (dim_normalize)
3. `_build_task_experts()` — GroupTaskExpertBasket or MLP fallback
4. `_build_hmm_projectors()` — 3 modes × projection
5. `_build_adatt()` — Adaptive Task Transfer
6. `_build_logit_transfer()` — 3-method dispatch, 5 edges
7. `_build_multidisciplinary_routing()` — 24D → 4 × 6D
8. `_build_task_towers()` — TowerRegistry (standard/contrastive)
9. `_build_evidential_layers()` — NIG for regression (config-gated)
10. `_build_sae()` — Sparse Autoencoder (config-gated)
11. `_build_task_loss_fns()` — `build_loss()` per task
12. `_build_loss_weighting()` — Uncertainty / GradNorm / DWA / Fixed

PLEInput 데이터 컨테이너

```
@dataclass
class PLEInput:
    features: Tensor                      # (batch, input_dim)
    feature_group_ranges: Optional[Dict] # group→(start,end) for routing
    cluster_ids: Optional[Tensor]         # (batch,) cluster assignment
    cluster_probs: Optional[Tensor]       # (batch, n_clusters) soft probs
    targets: Optional[Dict[str, Tensor]] # {task_name: label}
    hyperbolic_features: Optional[Tensor]   # (batch, 20)
    tda_features: Optional[Tensor]          # (batch, 70)
    collaborative_features: Optional[Tensor]# (batch, 64)
    hmm_journey: Optional[Tensor]           # (batch, 16)
    hmm_lifecycle: Optional[Tensor]         # (batch, 16)
    hmm_behavior: Optional[Tensor]          # (batch, 16)
    event_sequences: Optional[Tensor]       # (batch, seq_len, feat_dim)
    multidisciplinary_features: Optional[Tensor]  # (batch, 24)
    sample_weights: Optional[Tensor]        # (batch,)
```

부록: Config 파일 구조

시스템의 모든 파라미터는 2개 YAML 파일로 관리된다:

`pipeline.yaml`: 태스크 정의, 모델 구조, 학습 파라미터, AWS 인프라 설정

- `tasks`: 18개 태스크 (name, type, loss, loss_weight, label_col)
- `model.ple`: num_layers, extraction_dim, expert_basket
- `model.adatt`: task_groups, intra/inter strength
- `model.logit_transfers`: 5개 전이 엣지
- `training`: batch_size, epochs, learning_rate, amp

- `aws`: instance_type, spot, budget_limit

`feature_groups.yaml`: 12개 피처 그룹 정의

- `group_type`: transform (기존 컬럼 변환) | generate (Generator 호출)
- `generator`: Pool의 Generator 이름 참조
- `generator_params`: input_filter (dtype, exclude_binary, min_nunique 등)
- `target_experts`: 이 그룹의 피처를 받을 expert 목록
- `output_dim`: 출력 차원

도메인 전환 시 이 2개 파일만 교체하면 코드 수정 없이 전혀 다른 추천 시스템이 구성된다.